Robotic perception is a robot's ability to interpret sensory information like visual data to make decisions and interact with its environment. Deep learning (DL) applications are on the rise ever since robotics systems shifted towards autonomous intelligence from rule-based architectures. Unlike traditional computer vision methods, DL models learn features directly from data, enabling robust performance across diverse perception tasks. This review summarises findings from three surveys [1]–[3] to examine key robotic perception tasks—object recognition, semantic segmentation, human action recognition, and scene understanding—and the deep learning models used to address them.

2.a Object Recognition

Object recognition allows robots to identify and categorise objects in their environment, forming the foundation for tasks such as manipulation and navigation. Convolutional Neural Networks (CNNs) dominate this space due to their ability to extract spatial features directly from image data. The evolution from basic CNNs to deeper, optimised architectures like ResNet and MobileNet has significantly improved performance in robotic applications [1]. Transfer learning has also become an essential technique, enabling pre-trained models to adapt to robotic domains with limited labelled data [2]. However, deployment remains challenged by sensitivity to lighting conditions and occlusion [3].

2.b Semantic Segmentation

Semantic segmentation builds on object recognition by labeling each pixel of an image with a class label, enabling robots not only to recognize what objects are around them but where they are and how they are situated in space. Pixel-level perception is of key importance for tasks such as autonomous navigation, obstacle avoidance, and manipulation in unstructured environments. The early effectiveness of Fully Convolutional Networks (FCNs) for robotic segmentation tasks is widely realized, as they eliminated the necessity for fixed-size windows and enabled dense predictions [1]. Later architectures like U-Net and DeepLab have gained popularity in robotics with their spatial detail preservation through skip connections and multi-scale context fusion. [2] The architectures achieve strong accuracy but consume considerable computational resources. Approaches that merge segmentation with other tasks, such as 3D mapping or depth estimation, are increasingly popular [3].

2.c Human Action Recognition

Human action recognition enables robots to interpret and respond to dynamic human behaviours such as gestures, movements, and postures. Unlike static object classification, this task requires temporal modelling of visual data to capture motion patterns over time. Deep learning methods for this task often combine spatial feature extractors like CNNs with temporal models such as Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), or 3D CNNs that process video frames directly.[2] These hybrid architectures enables robots to recognise gestures, walking patterns, or specific human actions in real time. Action recognition models are increasingly tested in cluttered, real-world environments. Multi-modal perception that integrates visual and kinematic data is also gaining importance since it is vital for action recognition. [1] The future of robotic action understanding may lie in unified models capable of both perception and prediction, allowing anticipatory behaviours in human-robot collaboration. [3]

2.d Scene Understanding and Depth Estimation

Scene understanding provides the ability to interpret spatial context and layout as well as contextual relationships between objects, based on object location, depth, and layout. Structured scene interpretation relies heavily on depth perception and spatial reasoning, especially when using RGB-D inputs. Models based on attention and transforms proved effective tools for representing multi-object context and long-distance dependencies. Sensor fusion like fusing visual inputs with LiDAR, or inertial

measurements, has become commonplace for building robust environment models that facilitate navigation and planning in unstructured, dynamic environments [1]–[3].

3. Trends and Challenges

Deep learning has transformed the task of robotic perception in all its subareas from hand-engineered features towards end-to-end learning from data. There is an increasing trend towards multimodal perception, where the model combines RGB with depth, LiDAR, audio, or proprioceptive inputs for richer environmental perception. Multimodality helps the robot perform more robustly in cluttered, ambiguously textured environments. [3] Model efficiency remains a recurring thread, with increasing focus on the tradeoff between network depth and real-time execution, especially on mobile or embedded hardware. [1] Lightweight architectures such as MobileNet and methods such as model pruning and quantisation are becoming popular towards addressing the same.

Attention-based models, including Vision Transformers (ViTs), are also gaining ground for their ability to model spatial and contextual relationships, particularly in scene understanding. However, challenges remain. Deep models are data-hungry, and labelled robotic datasets are often scarce or domain-specific, limiting generalisation. Transfer learning can be effective but also risky if domains vary greatly. Ultimately, robustness and safety under uncertainty are still ongoing discussions, since models are susceptible to occlusion, low lighting, or adversarial perturbations. Deep learning systems in robotics that are data-efficient, interpretable, and trustworthy are needed to address these issues.

Robotic vision has made a fundamental change through the incorporation of deep learning, from task-specialized, narrow pipelines to flexible, data-driven architectures. From object detection to scene understanding, each of these subfields has been overhauled by increasingly advanced neural architectures, but challenges regarding efficiency, generalization, and robustness persist. Further research in these fields will be essential for making fully autonomous, context-aware robots possible.

Methodology:

For this project, I explored deep learning-based image classification using a convolutional neural network (CNN) trained using the CIFAR-100 dataset. The goal was to observe the effect of different hyperparameter choices on model performance on a task with high visual complexity and fine-grained object categories. Complex CIFAR-100 was better suited for making observations instead of simpler alternatives (e.g., MNIST, CIFAR-10) because it reflects real-world robotic vision challenges. It has low-resolution images with small inter-class variance and high intra-class noise.

The model followed a deep CNN architecture implemented in Keras. It consists of three convolutional blocks with increasing filter sizes (64, 128, 256), each using 3×3 kernels, ReLU activation, and followed by max pooling. Dropout layers were added after each block and the dense layer to regularise training. This configuration drew loosely from VGG-style networks, which are known for their strong performance on image classification while still being lightweight enough for experimentation without GPU acceleration. The final classifier includes a fully connected layer (512 units, ReLU) followed by a softmax layer for multi-class prediction over 100 classes. The network was trained using the Adam optimiser and categorical cross-entropy loss. I experimented with three hyperparameters in particular:

Learning rate: I tested 0.0001, 0.0005, and 0.001. These values were chosen based on common recommendations in CNN literature, where 0.001 is a standard starting point and lower values are often used to prevent overshooting on complex datasets like CIFAR-100. I hypothesised that a smaller learning rate would stabilise convergence across such a large label space.

Batch size: I tested 64, 128, and 256. Although smaller batch sizes promote more stable generalisation, they also increase the time of training. Batch size 128 has often been suggested as an effective trade-off in research articles, therefore I made it my baseline and tested smaller/larger batch sizes comparatively to explore their impact on gradient stability and test accuracy.

Dropout rate:I tested three different dropout rate setups (high, baseline, and low dropout) on convolutional and dense layers to see how stringently regularising the model would impact generalization on a dense visual dataset. CIFAR-100 was particularly suitable because it is susceptible to overfitting since there are only 500 training examples per class.

To improve generalisation further, data augmentation was applied during training using rotation (±15°), random shifts (±10%), and horizontal flipping. All experiments were run for 100 epochs, and each configuration was repeated 5 times to account for training variance. Metrics recorded include training/validation accuracy and loss, as well as final test set performance. Performance trends were visualised using accuracy and loss curves over epochs.

Code Availability
All training notebooks and code for the experiments described in this report are available at: https://drive.google.com/drive/folders/1CL_dEEAy3wxXM8SWNWYIbQ65sBgPC9K9?usp=sharing

Results:

Learning Rate:

Learning Rate	Train Accuracy	Test Accuracy	Train Loss	Test Loss
0.0001	0.5714 ± 0.0022	0.5673 ± 0.0011	1.6098 ± 0.0047	1.6029 ± 0.0006
0.0005	0.6016 ± 0.0092	0.6017± 0.0084	1.4777± 0.0381	1.4627± 0.0299
0.001	0.53 ± 0.0078	0.5283 ± 0.0069	1.802 ± 0.0427	1.7839 ± 0.0398

LR = 0.0005 produced the highest mean test accuracy and the most stable convergence across all trials.

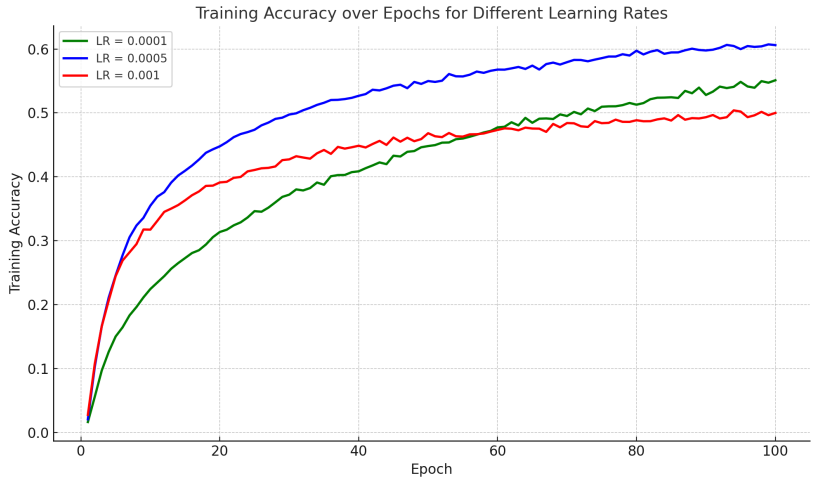


Figure X: Training accuracy curves for learning rates 0.0001, 0.0005, 0.001.

Figure X is the plot of learning accuracy for these three learning rates. It was evident that 0.0005 performed best because it achieves the highest accuracy and shows convergence although more epochs would be better to observe its convergence more. The lower learning rate of 0.0001 is slower in its learning so it ends in lower accuracy suggesting underfitting due to overly small weight updates. In more epochs it would reach higher values. 0.001 initially performed well, learning too fast, which causes the model to overshoot better solutions and reduce loss quickly. This, however; causes premature convergence. It settles into a suboptimal region too early. Further updates become ineffective and the accuracy stays low.

Batch Size

Batch Size	Train Accuracy	Test Accuracy	Train Loss	Test Loss
64	0.5805 ± 0.0060	0.582 ± 0.0060	1.5642 ± 0.0288	1.5502 ± 0.0237
128	0.6016 ± 0.0092	0.6017± 0.0084	1.4777± 0.0381	1.4627± 0.0299
256	0.6013 ± 0.0087	0.6038 ± 0.0094	1.4582 ± 0.0558	1.4561± 0.0514

The batch size of 128 yields the best overall results, achieving the highest test accuracy (0.6017 ± 0.0084) and a relatively low test loss (1.4627 ± 0.0299), suggesting it provides a strong balance between stable gradient updates and generalisation. While 256 achieves a slightly higher train accuracy (0.6013 ± 0.0087) and the lowest loss overall, it does not lead to a meaningful improvement in test accuracy and shows higher variance in test loss, indicating a risk of overfitting or less stable generalisation. The lowest batch size of 64 operates significantly less accurately and with higher loss, which may be due to noisier updates and slower convergence. These findings reflect typical batch size trade-offs: smaller batches have greater update variance, and extremely big batches have the possibility of degrading generalization performance. Overall, 128 appears to be the most effective and well-calibrated batch size for this task.

Dropout Rate

Dropout Rate	Train Accuracy	Test Accuracy	Train Loss	Test Loss
0.20/0.20/0.30/0.40 (low)	0.6049 ± 0.0045	0.6055 ± 0.0065	1.5307± 0.0238	1.5098± 0.0249
0.25/0.3/0.4/0.5 (baseline)	0.6016 ± 0.0092	0.6017± 0.0084	1.4777± 0.0381	1.4627± 0.0299
0.40/0.50/0.50/0.60 (high)	0.5501± 0.0078	0.5459± 0.0093	1.6926± 0.0495	1.6933± 0.0532

Dropout turned out to be the most volatile parameter in my experiments. Even minor changes to the dropout configuration affected both the model's ability to generalise and its variance across runs and provided examples for observing underfitting and overfitting. The values for average test and validation accuracies for low dropout rate slightly surpasses the baseline's accuracy values while the average test and validation loss for low dropout remains higher than the baseline loss values. This means model with low dropout made more confident but incorrect predictions. Looking at the loss plot for low dropout (Figure Y), it seems that after around epoch 30, validation loss began to surpass train loss and converges while train loss continues to decrease. This indicates overfitting, meaning the model fails to generalise and memorises training data instead. Conversely, the low accuracy rates for

high dropout indicate underfitting. So the baseline dropout rate stands out as the optimal trade-off between learning capacity and regularisation, avoiding both overfitting and underfitting.

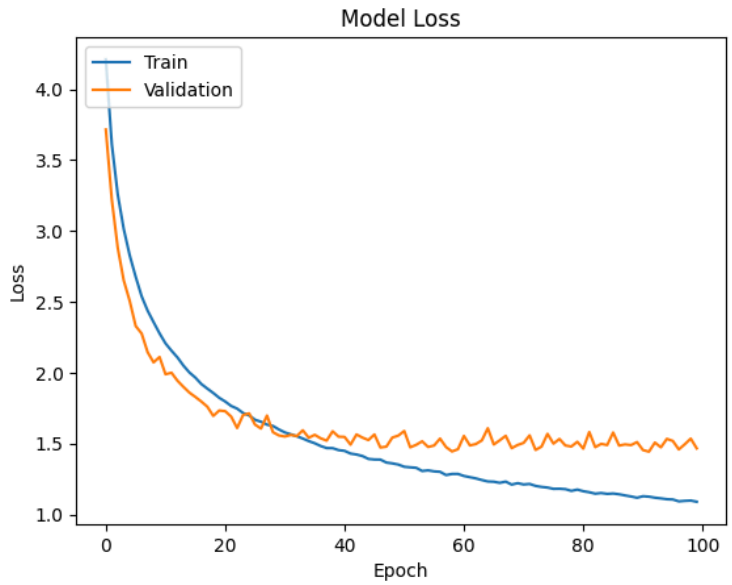


Figure Y: Epoch vs Loss for 100 epoch for the low dropout rate.

4. Future Steps

In some runs, models with similar accuracy had notably different losses—especially at the high and low learning rates. This suggests a mismatch between prediction certainty and correctness. I also noticed that for LR = 0.0001 and 0.0005, validation accuracy was not quite converging fully and was still on the rise. This also provided insight into epoch count as a hyperparameter although it was not on the chosen hyperparameter list. In this case a longer training , maybe 150 to 200 epochs, would allow model to even further improve. This would be a straightforward improvement for future runs. These experiments used one architecture on one dataset. To build on this, I could test other regularisation methods like CutMix or label smoothing, or try different learning rate schedules. Changing the optimiser might also improve stability. Running similar tests on other datasets would show whether these findings hold in different contexts.

References

[1] J. Ruiz-del-Solar, P. Loncomilla, and N. Soto, “A Survey on Deep Learning Methods for Robot Vision,” *arXiv preprint arXiv:1803.10862*, 2018.

[2] N. Manakitsa, G. S. Maraslidis, and L. Moysis, “A Review of Machine Learning and Deep Learning for Object Detection, Semantic Segmentation, and Human Action Recognition in Machine and Robotic Vision,” *Technologies*, vol. 12, no. 2, p. 15, Jan. 2024, doi: 10.3390/technologies12020015.

[3] R. F. N. Alshammari, H. Arshad, A. H. A. Rahman, and O. S. Albahri, “Robotics Utilization in Automatic Vision-Based Assessment Systems From AI Perspective: A Systematic Review,” *IEEE Access*, vol. 10, pp. 74321–74342, 2022.